



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Experiment No. 6
Implement Canonical Genetic Algorithm using binary representation.
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Aim: To implement the Canonical Genetic Algorithm (CGA) using binary representation for function maximization.

Theory:

A Canonical Genetic Algorithm (CGA) is an optimization technique inspired by natural selection. It operates using:

1. **Binary Encoding:** Solutions are represented as binary strings.
2. **Selection:** Choosing parents based on fitness.
3. **Crossover:** Combining parents to create offspring.
4. **Mutation:** Randomly flipping bits to maintain diversity.

Procedure/Algorithm:

1. Initialize a random population.
2. Evaluate fitness
3. Select parents using fitness-proportional selection.
4. Apply crossover and mutation.
5. Repeat for multiple generations.
6. Return the best solution.

Code:

```
import random

def fitness_function(x):
    """Fitness function to maximize  $f(x) = x^2$ """
    return x ** 2

def binary_to_decimal(binary_str):
    """Convert binary string to decimal number"""
    return int(binary_str, 2)

def generate_chromosome(length):
    """Generate a random binary chromosome"""
    return ''.join(random.choice('01') for _ in range(length))

def initialize_population(size, length):
    """Initialize a population with random chromosomes"""
    return [generate_chromosome(length) for _ in range(size)]

def select_parents(population):
    """Roulette Wheel Selection based on fitness"""
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

```
fitness_values = [fitness_function(binary_to_decimal(ind)) for ind in population]
total_fitness = sum(fitness_values)
probabilities = [f / total_fitness for f in fitness_values]
return random.choices(population, weights=probabilities, k=2)

def crossover(parent1, parent2):
    """Single-point crossover"""
    point = random.randint(1, len(parent1) - 1)
    offspring1 = parent1[:point] + parent2[point:]
    offspring2 = parent2[:point] + parent1[point:]
    return offspring1, offspring2

def mutate(chromosome, mutation_rate):
    """Bit Flip Mutation"""
    return "".join(bit if random.random() > mutation_rate else str(1 - int(bit)) for bit in chromosome)

def genetic_algorithm(pop_size, chrom_length, generations, mutation_rate):
    """Runs the Canonical Genetic Algorithm (CGA)"""
    population = initialize_population(pop_size, chrom_length)
    best_solution = None
    best_fitness = float('-inf')

    for gen in range(generations):
        new_population = []
        for _ in range(pop_size // 2):
            parent1, parent2 = select_parents(population)
            offspring1, offspring2 = crossover(parent1, parent2)
            offspring1, offspring2 = mutate(offspring1, mutation_rate), mutate(offspring2, mutation_rate)
            new_population.extend([offspring1, offspring2])
        population = new_population
        best_in_gen = max(population, key=lambda ind: fitness_function(binary_to_decimal(ind)))
        best_in_gen_fitness = fitness_function(binary_to_decimal(best_in_gen))
        if best_in_gen_fitness > best_fitness:
            best_solution = best_in_gen
            best_fitness = best_in_gen_fitness
        print(f"Generation {gen+1}: Best = {best_solution}, Value = {binary_to_decimal(best_solution)}, Fitness = {best_fitness}")

    return best_solution, binary_to_decimal(best_solution), best_fitness

POPULATION_SIZE = int(input("Enter population size: "))
CHROMOSOME_LENGTH = int(input("Enter chromosome length: "))
GENERATIONS = int(input("Enter number of generations: "))
MUTATION_RATE = float(input("Enter mutation rate (e.g., 0.01): "))
best_chromosome, best_value, best_fitness = genetic_algorithm(POPULATION_SIZE,
CHROMOSOME_LENGTH, GENERATIONS, MUTATION_RATE)
print(f"Best Solution: {best_chromosome} -> Value: {best_value}, Fitness: {best_fitness}")
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

2024-2025

Output:

```
Enter population size: 10
Enter chromosome length: 8
Enter number of generations: 10
Enter mutation rate (e.g., 0.01): 0.01
Generation 1: Best = 11100101, Value = 229, Fitness = 52441
Generation 2: Best = 11111110, Value = 254, Fitness = 64516
Generation 3: Best = 11111110, Value = 254, Fitness = 64516
Generation 4: Best = 11111110, Value = 254, Fitness = 64516
Generation 5: Best = 11111110, Value = 254, Fitness = 64516
Generation 6: Best = 11111110, Value = 254, Fitness = 64516
Generation 7: Best = 11111110, Value = 254, Fitness = 64516
Generation 8: Best = 11111110, Value = 254, Fitness = 64516
Generation 9: Best = 11111110, Value = 254, Fitness = 64516
Generation 10: Best = 11111110, Value = 254, Fitness = 64516
Best Solution: 11111110 -> Value: 254, Fitness: 64516
```

Conclusion: The Canonical Genetic Algorithm efficiently finds optimal solutions by evolving a population through selection, crossover, and mutation.

Answer the following questions:

1. What is the purpose of using a binary representation in this experiment?

Binary representation is used in the Canonical Genetic Algorithm (CGA) because it is simple, efficient, and easy to manipulate using genetic operators like crossover and mutation. Each candidate solution is represented as a string of 0s and 1s, which makes it easy to apply bitwise operations. Binary encoding also ensures a uniform search space, allowing the algorithm to explore possible solutions effectively. Additionally, binary representation is commonly used in CGA because it aligns well with the classic genetic algorithm structure, making operations like selection, mutation, and crossover straightforward to implement.

2. How does mutation impact the evolution of solutions in this experiment?

In this experiment, mutation helps maintain diversity in the population by flipping random bits in the binary chromosomes. This prevents the algorithm from getting stuck in local optima by introducing new genetic variations. Without mutation, all individuals might become too similar over generations, reducing the chances of finding better solutions. By randomly changing some bits, mutation ensures that the algorithm continues to explore different parts of the solution space, improving the chances of discovering the best possible solution.